

## Lab 12 Arrays (one dimension) II

### Objective

- To practically familiarize student with the concept of array and related operations performed on array.

### Current Lab Learning Outcomes (LLO)

As a second Lab on Chapter 7, this lab continues practicing the Chapter 7 theory with students, with using ideas of arrays. By completion of the lab the students should be able to

- how to pass an array to method
- how to return an array from the methods

### Lab Requirements

NetBeans IDE 8.0.1 or higher

### Lab Assessment/Activities

#### Practice Activity with Lab Instructor (10 minutes)

**(Find the smallest element)** Write a method that finds the smallest element in an array of double values using the following header:

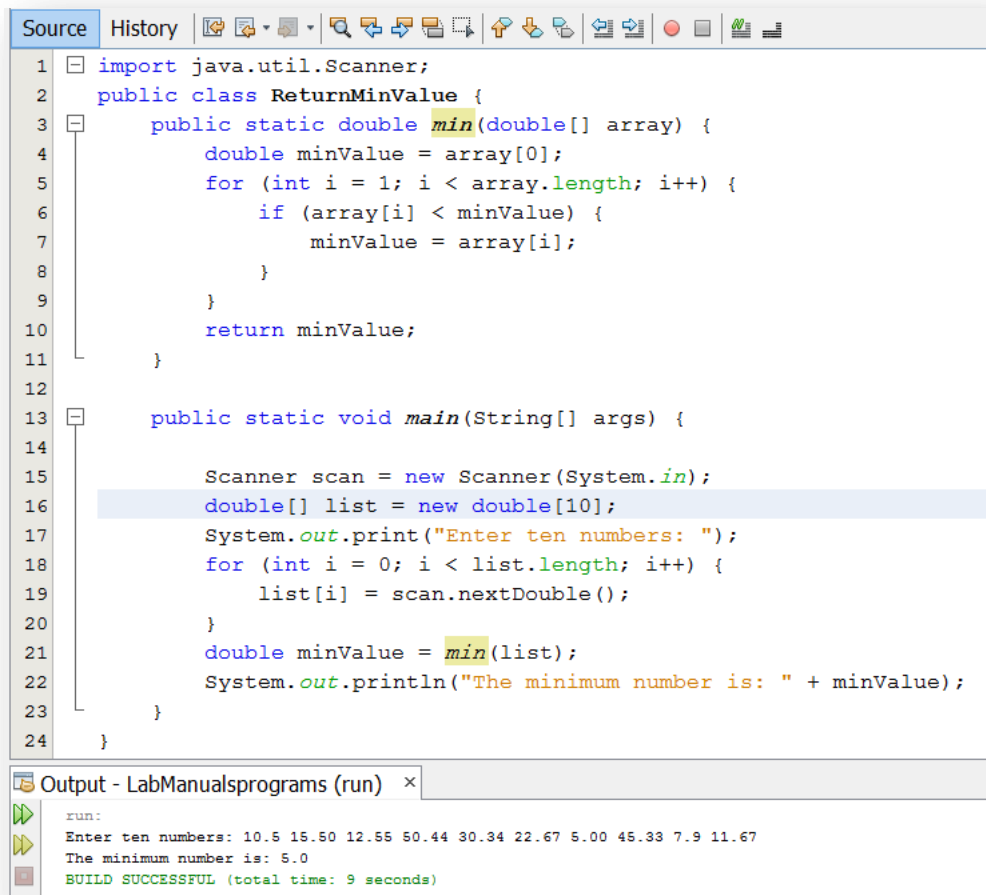
```
public static double min(double[] array)
```

Write a test program that prompts the user to enter ten numbers, invokes this method to return the minimum value, and displays the minimum value. Here is a sample run of the program:

```
Enter ten numbers: 1.9 2.5 3.7 2 1.5 6 3 4 5 2 Enter
The minimum number is: 1.5
```

**Solution:**

1. Open NetBeans and create a new project
2. Create a new java main class and write its name as **ReturnMinValue**
3. Write the following code inside the class, Figure (1).



```
1 import java.util.Scanner;
2 public class ReturnMinValue {
3     public static double min(double[] array) {
4         double minValue = array[0];
5         for (int i = 1; i < array.length; i++) {
6             if (array[i] < minValue) {
7                 minValue = array[i];
8             }
9         }
10        return minValue;
11    }
12
13    public static void main(String[] args) {
14
15        Scanner scan = new Scanner(System.in);
16        double[] list = new double[10];
17        System.out.print("Enter ten numbers: ");
18        for (int i = 0; i < list.length; i++) {
19            list[i] = scan.nextDouble();
20        }
21        double minValue = min(list);
22        System.out.println("The minimum number is: " + minValue);
23    }
24 }
```

Output - LabManualsprograms (run) x

run:  
Enter ten numbers: 10.5 15.50 12.55 50.44 30.34 22.67 5.00 45.33 7.9 11.67  
The minimum number is: 5.0  
BUILD SUCCESSFUL (total time: 9 seconds)

**Figure 1:** The program to read 10 numbers and using method to return the minimum numbrers

**Short Exercise** (5 minutes)

**(Find the largest index of the small element)** Rewrite the above method *min* to return the largest index of the smallest element.

**Individual Activities:** (60 minutes)

1. **(Average an array)** Write two overloaded methods that return the average of an array with the following headers:

```
public static int average(int[] array)
public static double average(double[] array)
```

Write a test program that prompts the user to enter ten double values, invokes this method, and displays the average value.

2. **(Sorted?)** Write the following method that returns true if the list is already sorted in increasing order.

```
public static boolean isSorted(int[] list)
```

Write a test program that prompts the user to enter a list and displays whether the list is sorted or not. Here is a sample run. Note that the first number in the input indicates the number of the elements in the list. This number is not part of the list.

```
Enter list: 8 10 1 5 16 61 9 11 1 ↵ Enter
The list is not sorted
```

```
Enter list: 10 1 1 3 4 4 5 7 9 11 21 ↵ Enter
The list is already sorted
```

3. **(Eliminate duplicates)** Write a method that returns a new array by eliminating the duplicate values in the array using the following method header:

```
public static int[] eliminateDuplicates(int[] list)
```

Write a test program that reads in ten integers, invokes the method, and displays the result. Here is the sample run of the program:

```
Enter ten numbers: 1 2 3 2 1 6 3 4 5 2 ↵ Enter
The distinct numbers are: 1 2 3 6 4 5
```

4. **(Game: hangman)** Write a hangman game that randomly generates a word and prompts the user to guess one letter at a time, as shown in the sample run. Each letter in the word is displayed as an asterisk. When the user makes a correct guess, the actual letter is then displayed. When the user finishes a word, display the number of misses and ask the user whether to continue to play with another word. Declare an array to store words, as follows:

```
// Add any words you wish in this array
```

```
String[] words = {"write", "that", ...};
```

```
(Guess) Enter a letter in word ***** > p Enter
(Guess) Enter a letter in word p***** > r Enter
(Guess) Enter a letter in word pr**r** > p Enter
p is already in the word
(Guess) Enter a letter in word pr**r** > o Enter
(Guess) Enter a letter in word pro*r** > g Enter
(Guess) Enter a letter in word progr** > n Enter
n is not in the word
(Guess) Enter a letter in word progr** > m Enter
(Guess) Enter a letter in word progr*m > a Enter
The word is program. You missed 1 time
Do you want to guess another word? Enter y or n>
```

5. **(Pattern recognition: consecutive four equal numbers)** Write the following method that tests whether the array has four consecutive numbers with the same value.

```
public static boolean isConsecutiveFour(int[] values)
```

Write a test program that prompts the user to enter a series of integers and displays if the series contains four consecutive numbers with the same value. Your program should first prompt the user to enter the input size—i.e., the number of values in the series. Here are sample runs:

```
Enter the number of values: 8 Enter
Enter the values: 3 4 5 5 5 5 4 5 Enter
The list has consecutive fours
```

```
Enter the number of values: 9 Enter
Enter the values: 3 4 5 5 6 5 5 4 5 Enter
The list has no consecutive fours
```

## Lab Description

### Passing arrays to method

- Java uses pass by value to pass arguments to a method. There are important differences between passing a value of variables of primitive data types and passing arrays.
- For a parameter of a primitive type value, the actual value is passed. Changing the value of the local parameter inside the method does not affect the value of the variable outside the method.
- For a parameter of an array type, the value of the parameter contains a reference to an array; this reference is passed to the method. Any changes to the array that occur inside the method body will affect the original array that was passed as the argument.

### Example 1

Method that accepts an array and prints its values

```
public static void printArraay(int [] a)
{
    for(int i=0;i<a.length;i++)
        System.out.print(a[i]+ " ");
}
```

### Invoke the method by passing an array

```
int []arr= {10,20,15,30,50};
printArraay(arr);
```

### Example 2

Method that enters values into array and returns it.

```
public static int []readArray()  
{  
    Scanner sc=new Scanner(System.in);  
    System.out.print("Enter number of elements: ");  
    int numValues=sc.nextInt();  
    int []arr=new int[numValues];  
    System.out.println("Enter "+ numValues+ " elements ");  
    for(int i=0;i<numValues;i++)  
        arr[i]=sc.nextInt();  
    return arr;  
}
```

Invoke the method and receive the returned array

```
int []list=readArraay();
```

### **Extra (supplementary) Materials** (if any)

None

---